```
public class ReferenceInvestigator {

    public static void main(String[] args) {


        System.out.println("--- 1. Primitive Overflow Demonstration ---");
        byte maxByte = 90;
        System.out.println("Initial byte value: " + maxByte);


        while (maxByte > 0) {
            maxByte++;
        }
        System.out.println("Byte after overflowing via increment (unexpected result): " + maxByte);
        System.out.println();



        System.out.println("--- 2 & 3. String Comparison & Reference Check ---");
        String s1 = new String("Investigator");
        String s2 = new String("Investigator");
        String s3 = "Investigator";

        // Comparing s1 and s2
        boolean s1_eq_s2_ref = (s1 == s2);
        boolean s1_eq_s2_val = s1.equals(s2);
        System.out.println("s1 == s2: " + s1_eq_s2_ref);
        System.out.println("s1.equals(s2): " + s1_eq_s2_val);

        // Comparing s1 and s3
        boolean s1_eq_s3_ref = (s1 == s3);
        boolean s1_eq_s3_val = s1.equals(s3);
        System.out.println("s1 == s3: " + s1_eq_s3_ref);
        System.out.println("s1.equals(s3): " + s1_eq_s3_val);

        // Comparing s2 and s3
        boolean s2_eq_s3_ref = (s2 == s3);
        boolean s2_eq_s3_val = s2.equals(s3);
        System.out.println("s2 == s3: " + s2_eq_s3_ref);
        System.out.println("s2.equals(s3): " + s2_eq_s3_val);
        System.out.println();

        /*
         * EXPLANATION OF '==' COMPARISONS
```

```
        *
        * 1. 's1 == s2' is FALSE:
        *    The 'new' keyword explicitly forces the JVM to allocate two distinct
        *    objects at different memory locations in the heap. Since '==' checks
        *    memory addresses (reference equality), it returns false.
        *
        * 2. 's1 == s3' is FALSE:
        *    's1' points to a specific object explicitly created in the heap, while
        *    's3' is a string literal stored in the String Constant Pool. Since they
        *    point to entirely different locations in memory, '==' returns false.
        *
        * 3. 's2 == s3' is FALSE:
        *    For the exact same reason as above; 's2' resides at a unique heap address
        *    created by 'new String()', whereas 's3' points directly to the pool.
        */


        System.out.println("--- 4. Shared Array Reference Demonstration ---");
        int[] originalArray = {5, 10, 15};


        int[] sharedArray = originalArray;

        System.out.println("Before modification via originalArray:");
        System.out.println("originalArray: " + java.util.Arrays.toString(originalArray));
        System.out.println("sharedArray:   " + java.util.Arrays.toString(sharedArray));

        originalArray[0] = 777;

        System.out.println("\nAfter modification via originalArray:");
        System.out.println("originalArray: " + java.util.Arrays.toString(originalArray));
        System.out.println("sharedArray:   " + java.util.Arrays.toString(sharedArray) + "
(Modification is visible here!)");
    }
}
```

OUTPUT GENERATED FILES DEBUGGER GUI

```
--- 1. Primitive Overflow Demonstration ---
Initial byte value: 90
Byte after overflowing via increment (unexpected result): -128

--- 2 & 3. String Comparison & Reference Check ---
s1 == s2: false
s1.equals(s2): true
s1 == s3: false
s1.equals(s3): true
s2 == s3: false
s2.equals(s3): true

--- 4. Shared Array Reference Demonstration ---
Before modification via originalArray:
originalArray: [5, 10, 15]
sharedArray:   [5, 10, 15]

After modification via originalArray:
originalArray: [777, 10, 15]
sharedArray:   [777, 10, 15] (Modification is visible here!)
```